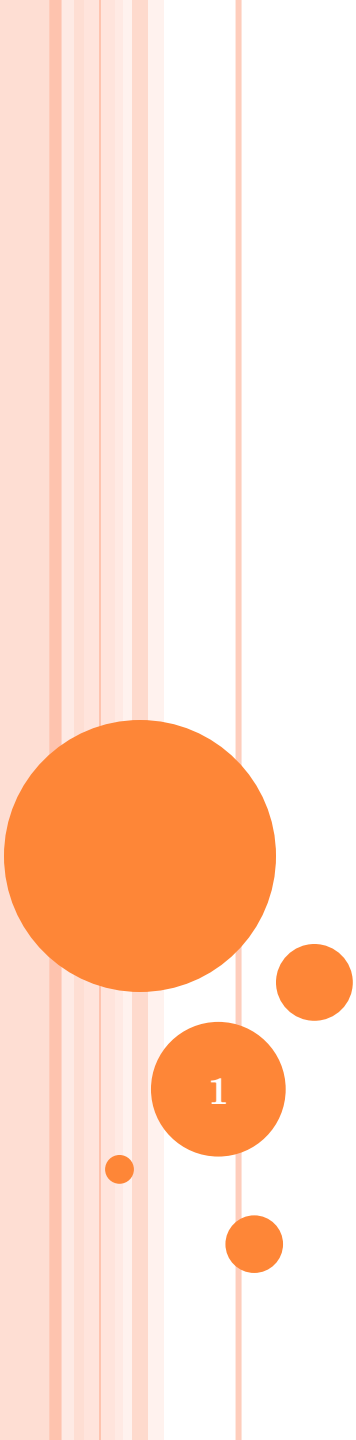




SISTEME DE OPERARE

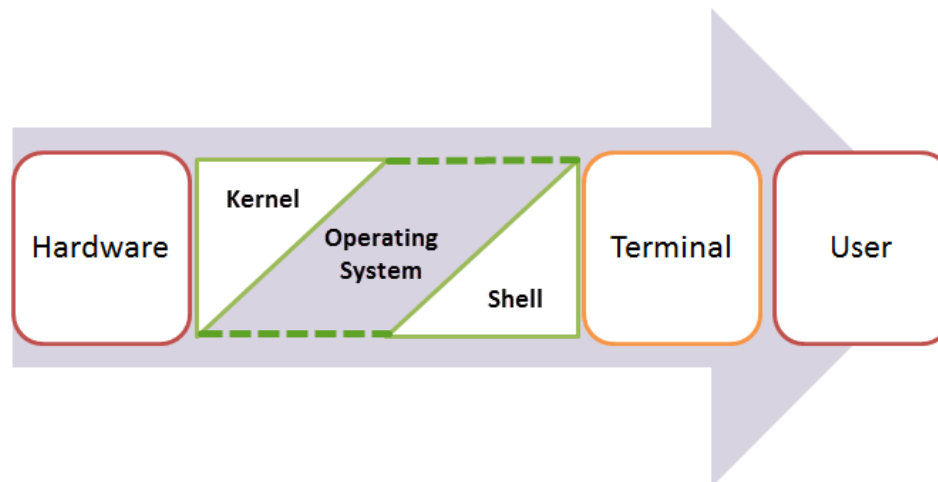
**Curs 2: Interpretorul de comenzi - shell,
Interacțiunea programelor cu SO,
Programare în C – recomandări și obs**

Kinga Márton

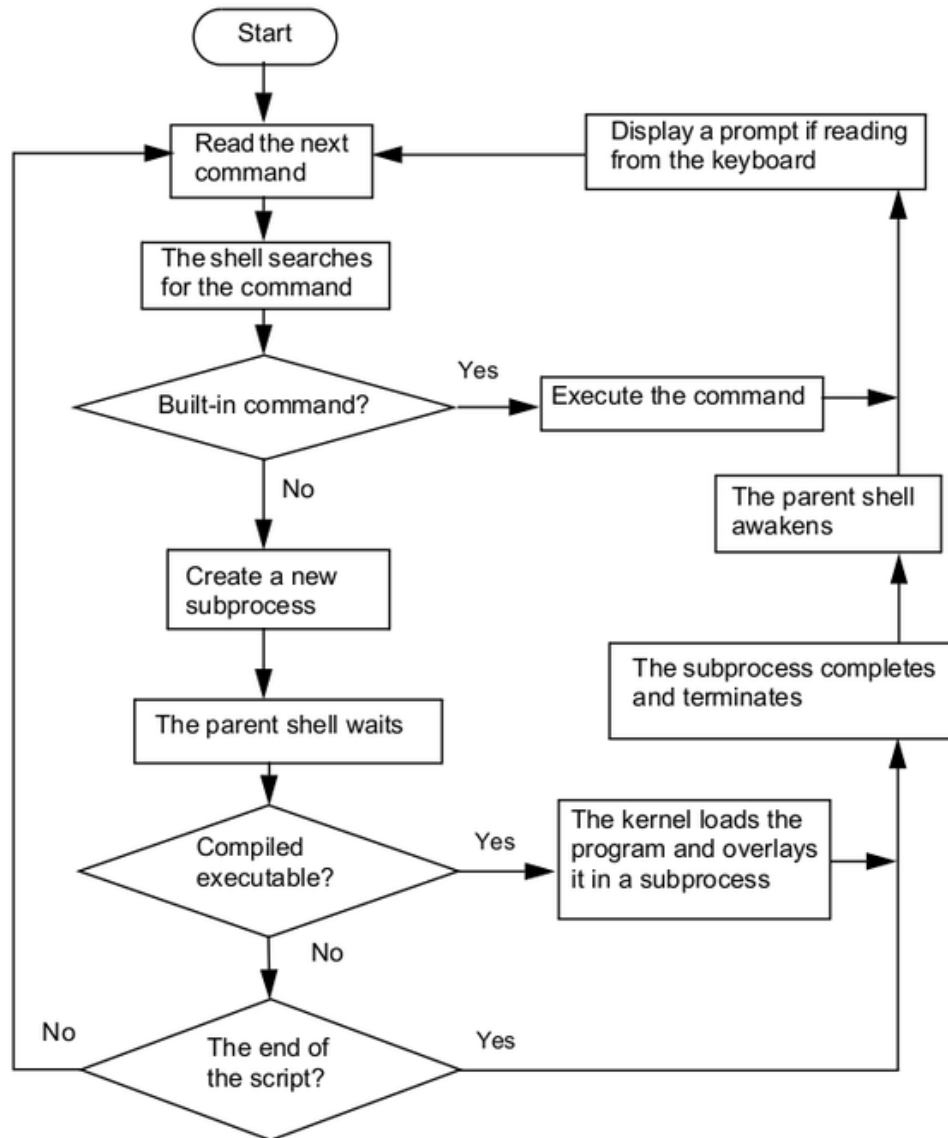
**Universitatea Tehnică din Cluj-Napoca
Departamentul de Calculatoare**

INTERPRETORUL DE COMENZI – SHELL-UL

- Shell-ul este o interfață între utilizator și SO
 - Aparține parțial SO
 - Rulează în spațiul utilizator
 - Specific sistemului de operare
 - Tipuri
 - Interfață text – interpretorul de comenzi din linia de comandă
 - Interfață grafică
- Permite interacțiunea cu SO
 - Comenzile introduse de utilizator sunt predate SO spre execuție
 - Lansarea altor programe și aplicații



SHELL - FUNCTIONALITATE



SHELL - COMENZI

○ Comenzi

- **Interne** (builtin): gestionate și executate de shell (cd, echo, ...)
- **Externe**: caută și rulează fișierul executabil cu numele comenzii
 - Specificate ca nume
 - Ex. ls, grep, wc, passwd,
 - Caută executabilul în directoarele din variabila de mediu PATH
 - Ex. /usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
 - Alege primul fișier găsit cu numele dat – ordinea contează!
 - Specificate ca și cale
 - Ex. /bin/ls, ./hello.exe
- Forma generală
nume_comanda -optiuni lista_argumente

SHELL – MOD DE EXECUȚIE

○ Execuție sincronă

- Modul implicit
- Shell-ul așteaptă terminarea comenzii curente, înainte să reafișeze prompterul și să accepte alte comenzi
 - Ex. Deschiderea editorului text gedit din terminal (`gedit fisier`)
 - Terminalul este suspendat până la închiderea aplicației gedit

○ Execuție asincronă – în background

- Activată prin specificarea simbolului "&" la finalul comenzii
- Shell-ul nu așteaptă terminarea comenzii curente, ci revine imediat să afișeze prompterul și să preia alte comenzi
 - Ex. Deschiderea gedit în fundal – background (`gedit fisier &`)
 - Terminalul permite preluarea de noi comenzi imediat

SHELL – NUMELE COMENZILOR

- Calea către executabil

- Ex. /bin/ls
 ./hello

- Numele

- Ex. ls, pwd, passwd
- Dacă numele comenzii este doar un nume (nu cale de acces)
 - Caută în directoarele specificate în variabila de mediu PATH
 - echo \$PATH
 - Pentru a vedea conținutul variabilei PATH
 - Ordinea este importantă! Se execută prima apariție în lista de directoare din PATH

VARIABLE DE MEDIU

- Listă de perechi **nume** – **valoare** care descriu mediul de execuție al aplicației
 - Comanda env – listează toate variabilele de mediu cu valorile lor
 - Ex. PATH, HOME, USER, PWD
- Se moștenesc de la procesul părinte
 - Ex. Lansez din shell un alt shell sau un executabil C
- Acces din linia de comandă / scripturi
 - Setarea valorii: VAR=VALOARE
 - Atenție, nu punem spațiu înainte sau după =
 - Setarea atributului de moștenire
 - export VAR
 - Desetarea atributului de moștenire
 - export -n VAR
 - Eliminarea unei variabile din mediu
 - unset VAR
 - Acces prin prefixarea caracterului "\$"
 - echo \$PATH

VARIABILE DE MEDIU

○ Acces din programul C

```
#include <stdio.h>

// Most of the C compilers support a third parameter to main
// which stores all environment variables
int main(int argc, char* argv[], char* envp[])
{
    int i;

    for (i = 0; envp[i] != NULL; i++)
    {
        printf("\nenv[%d]: %s", i, envp[i]);
    }

    return 0;
}
```

```
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char* argv[], char* envp[])
{
    char* home, * host;

    home = getenv("HOME");
    host = getenv("HOSTNAME");

    printf("Your home directory is %s on %s.\n", home, host);

    return 0;
}
```

FIȘIERE DE COMENZI

- Numite *shell script* (extensia .sh)
 - Fișiere text
 - Conțin comenzi pt shell + cuvinte cheie + funcții + instrucțiuni
 - Comenzi cu sintaxa identică cu cea folosită în linia de comandă
 - Acceptă argumente
- Modalitate practică pentru
 - Stocarea comenzilor pt rulare de mai multe ori
 - Rulare cu set diferit de argumente
 - Automatizarea acțiunilor de gestiune în sistem
 - Alternativă la execuția interactivă (la comenzi date succesiv din linia de comandă)
- Comenzile sunt interpretate (nu compilate)

FIȘIERE DE COMENZI

○ Exemple

```
#!/bin/bash
# This is a comment!
echo Hello World          # This is a comment, too!
```

```
$ chmod 755 first.sh
$ ./first.sh
Hello World
$
```

```
#!/bin/bash
echo What is your name?
read MY_NAME
echo "Hello $MY_NAME - hope you're well."
```

```
#!/bin/bash
echo "What is your name?"
read USER_NAME
echo "Hello $USER_NAME"
echo "I will create you a file called ${USER_NAME}_file"
touch "${USER_NAME}_file"
```

PARAMETRI DIN LINIA DE COMANDĂ

○ Variabile speciale

- \$# numărul argumentelor din linia de comandă (exclusiv \$0)
- \$@,\$* lista parametrilor poziționali
- \$0 numele scriptului
- \$1, \$2, ..., \${10} parametri poziționali (argumentele efective)

```
echo "Get cmd line args one-by-one."  
echo "Works for args with spaces."  
for i  
do  
    echo $i  
done
```

```
./args1.sh 1 2 '3 4' 5  
1  
2  
3 4  
5
```

```
echo "Get cmd line args one-by-one."  
echo "Works for args with spaces."  
for i in "$@"  
do  
    echo "$i"  
done
```

```
./args1.sh 1 2 '3 4' 5  
1  
2  
3 4  
5
```

```
echo "Get cmd line args one-by-one."  
echo "Doesn't work for args with spaces."  
for i in $*  
do  
    echo "$i"  
done
```

```
./args1.sh 1 2 '3 4' 5  
1  
2  
3  
4  
5
```

```
#!/bin/bash
# List all elements in a directory

# Check the number of parameters
if test $# -ne 1
then
    echo USAGE: $0 dir
    exit
fi

working_dir=$1

# Check if the given parameter is a directory
if test ! -d $working_dir
then
    echo The parameter must be a directory
    exit
fi

echo 'Get all visible (unhidden) elements in the dir'
for elem in $working_dir/*
do
    if test -f "$elem"
    then
        echo File: $elem
    elif test -d "$elem"
    then
        echo Dir: $elem
    fi
done
```

FIȘIERE DE COMENZI

○ Exemple

```
for elem in *  
do  
    echo $elem  
done
```

Echivalent cu **ls**

```
for elem in /home/so/*  
do  
    echo $elem  
done
```

Echivalent cu **ls /home/so**

```
for elem in "/home/so/*" "/home/so/*."  
do  
    echo $elem  
done
```

Echivalent cu **ls -a /home/so**

```
while read line  
do  
    echo $line  
done < file_name
```

FIȘIERE DE COMENZI

○ Exemple

```
#!/bin/bash

a=0
while [ "$a" -lt 10 ] # this is loop1
do
    b=$a
    while [ "$b" -ge 0 ] # this is loop2
    do
        echo -n "$b "
        b=$(( b - 1 ))
    done

    a=$(( a + 1 ))
    echo
done
```

```
0
1 0
2 1 0
3 2 1 0
4 3 2 1 0
5 4 3 2 1 0
6 5 4 3 2 1 0
7 6 5 4 3 2 1 0
8 7 6 5 4 3 2 1 0
9 8 7 6 5 4 3 2 1 0
```

FIȘIERE DE COMENZI

- Studiați mai multe despre scripturi în lucrarea de lab nr 2
- Alte tutoriale de pe net 😊
 - <https://www.shellscript.sh/>
 - https://www.tutorialspoint.com/unix/shell_scripting.htm